

BÀI THỰC HÀNH SỐ 4



Content

1. Lab 01 – Load LLM và hỏi đáp cơ bản
2. Lab 02 – Các kỹ thuật prompting
3. Lab 03 – Ứng dụng bài toán Hỏi đáp về Luật.

1. Load LLM và hỏi đáp cơ bản

- HuggingFace: nền tảng cộng đồng mã nguồn mở chuyên chia sẻ model, dataset, spaces...
- Load Large Language Model từ HuggingFace:
 - Tạo Access Token trên HuggingFace
 - Login bằng access token

```
from google.colab import userdata  
hf_token = userdata.get("HUGGINGFACE_TOKEN")  
from huggingface_hub import login  
login(token=hf_token)
```

- Load model LLM hỏi đáp thường dùng:
 - AutoModelForCausalLM
 - AutoModelFor

2. Các kỹ thuật prompting

- Zero-shot Prompting
- Few-shot Prompting
- Instruction Prompting
- Chain-of-Thought (CoT) Prompting
- Tree-of-Thought (ToT) Prompting

Zero-shot Prompting

2.1 Định nghĩa

- Mô hình AI được giao một nhiệm vụ mà không có bất kỳ ví dụ nào
- Phải hoàn toàn dựa vào kiến thức đã được huấn luyện trước và các nguồn truy xuất được để tạo ra một phản hồi.

Zero-shot Prompting

2.2 Example

Viết một đoạn mã Python đơn giản để điều khiển chuyển động nhân vật trong một game platform 2D

Dưới đây là một đoạn mã Python đơn giản sử dụng thư viện `pygame` để điều khiển chuyển động của một nhân vật trong game platform 2D. Nhân vật có thể di chuyển sang trái, phải và nhảy:

```
python

import pygame
import sys

# Khởi tạo pygame
pygame.init()

# Thiết lập kích thước màn hình
WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Simple Platformer")

# Màu sắc
WHITE = (255, 255, 255)
BLUE = (0, 0, 255)

# Thiết lập nhân vật
player_width, player_height = 50, 50
player_x, player_y = 100, HEIGHT - player_height - 50
player_velocity = 5
jump_speed = 15
gravity = 1
is_jumping = False
vertical_velocity = 0

# Clock để giới hạn FPS
clock = pygame.time.Clock()

# Vòng lặp game
```

Few-shot Prompting

3.1 Định nghĩa

- AI được cung cấp một vài ví dụ trước khi được yêu cầu tạo ra một phản hồi. Những ví dụ này giúp AI nhận diện ngữ cảnh và cải thiện độ chính xác.

Fewshot-shot Prompting

3.2 Example

Nhiệm vụ: Gợi ý engine lập trình game phù hợp nhất cho từng thể loại game.

Ví dụ 1:

Thể loại game: Platformer 2D

Engine đề xuất: Godot

Lý do: Nhẹ, dễ học, hỗ trợ 2D xuất sắc, mã nguồn mở.

Ví dụ 2:

Thể loại game: Game nhập vai 3D cao cấp

Engine đề xuất: Unreal Engine

Lý do: Đồ họa chất lượng AAA, lập trình trực quan với Blueprint, hệ thống xử lý tài nguyên mạnh mẽ.

Ví dụ 3:

Thể loại game: Game giải đố di động đơn giản

Engine đề xuất: Unity

Lý do: Hỗ trợ đa nền tảng, có cửa hàng tài nguyên lớn, dễ tích hợp quảng cáo và kiếm tiền.

Giờ hãy gợi ý engine cho: Trải nghiệm game kinh dị thực tế ảo (VR)

Thể loại game: Trải nghiệm game kinh dị thực tế ảo (VR)

Engine đề xuất: Unreal Engine

Lý do: Hỗ trợ VR mạnh mẽ, đồ họa chân thực phù hợp với không khí kinh dị, công cụ âm thanh và ánh sáng tiên tiến giúp tạo cảm giác nhập vai sâu sắc, cùng với khả năng tích hợp các thiết bị VR như Oculus Rift, HTC Vive, v.v.

Instruction Prompting

4.1 Định nghĩa

- AI nhận được hướng dẫn rõ ràng, trực tiếp để thực hiện một nhiệm vụ.
- Hoạt động hiệu quả khi sử dụng lệnh rõ ràng thay vì dựa vào ví dụ.

Instruction Prompting

4.2 Example

Nhiệm vụ: Tối ưu hiệu suất cho một game 2D trên thiết bị di động.

Hướng dẫn:

- 1.Xác định các yếu tố chính ảnh hưởng đến hiệu suất của game (ví dụ: render, logic xử lý, tài nguyên...).
- 2.Đề xuất các cải tiến kỹ thuật để giảm lag và tăng tốc độ khung hình mà vẫn giữ được chất lượng trải nghiệm.
- 3.Đưa ra các khuyến nghị về công cụ hoặc kỹ thuật profiling để kiểm tra và theo dõi hiệu suất trong quá trình phát triển.

1. Yếu tố chính ảnh hưởng đến hiệu suất

- Render (vẽ màn hình):
 - Quá nhiều đối tượng vẽ cùng lúc (sprites, hiệu ứng...).
 - Dùng ảnh kích thước lớn không cần thiết.
 - Thiếu batch rendering (vẽ theo nhóm).
- Logic xử lý:
 - Cập nhật tất cả vật thể mỗi khung hình, kể cả khi chúng không hiển thị.
 - Các thuật toán AI hoặc xử lý vật lý quá phức tạp.
- Tài nguyên (Resource):
 - Âm thanh/ảnh chưa nén đúng cách.
 - Tải quá nhiều dữ liệu vào RAM cùng lúc.
 - Quản lý bộ nhớ kém gây rò rỉ hoặc phân mảnh.
- Tương tác người dùng:
 - Phản hồi đầu vào chậm.
 - UI sử dụng hiệu ứng động nặng.

2. Đề xuất cải tiến kỹ thuật

- 🎨 **Tối ưu vẽ (Rendering):**
 - Dùng Sprite Atlas (Texture Atlas): giảm số lần gọi draw call.
 - Loại bỏ các đối tượng ngoài màn hình: chỉ render những gì trong khung nhìn.
 - Giảm tần suất cập nhật hiệu ứng động: ví dụ: cập nhật animation theo thời gian thực, không phải mỗi frame.
 - Dùng shader đơn giản: tránh hiệu ứng hậu kỳ hoặc blending phức tạp.
- 🧠 **Tối ưu logic:**
 - Cập nhật đối tượng theo vùng hiển thị (Culling): xử lý logic chỉ với những đối tượng có khả năng tương tác.
 - Giảm tần suất tính toán vật lý/AI: dùng fixed timestep hoặc coroutine/timer.
 - Tránh tạo/destroy đối tượng liên tục: dùng object pooling.
- 📦 **Tối ưu tài nguyên:**
 - Sử dụng định dạng ảnh tối ưu cho di động: như WebP, PNG8.
 - Tối ưu kích thước âm thanh: nén bằng Ogg Vorbis hoặc AAC.
 - Giải phóng tài nguyên không dùng đến: đặc biệt là khi chuyển scene.

Prompt

Response



A14LIFE

Chain-of-Thought (CoT) Prompting

5.1 Định nghĩa

- Chain-of-Thought (CoT) Prompting khuyến khích AI phân tích các vấn đề phức tạp thành từng bước lý luận trước khi đưa ra câu trả lời cuối cùng.
- Giúp cải thiện độ chính xác cho các nhiệm vụ ra quyết định logic, số liệu, hoặc nhiều bước.

Chain-of-Thought (CoT) Prompting

5.2 Example

Task: Calculate the total cost for a bulk fabric order, considering different cost factors.

Example:

Fabric: 100% Organic Cotton

Order Quantity: 10,000 meters

Base Price: \$5 per meter

Additional Costs:

- Dyeing: \$0.50 per meter
- Special Finishing: \$0.75 per meter
- Shipping: Flat \$1,000

Step-by-step calculation:

1. Base cost = $10,000 \times \$5 = \$50,000$
2. Dyeing cost = $10,000 \times \$0.50 = \$5,000$
3. Finishing cost = $10,000 \times \$0.75 = \$7,500$
4. Total before shipping = $\$50,000 + \$5,000 + \$7,500 = \$62,500$
5. Final cost = $\$62,500 + \$1,000 = **\$63,500**$

Now calculate for:

Fabric: 80% Polyester, 20% Cotton

Order Quantity: 8,000 meters

Base Price: \$4.50 per meter

Additional Costs:

- Dyeing: \$0.40 per meter
- Special Finishing: \$0.60 per meter
- Shipping: Flat \$800

Let's calculate the total cost for the bulk fabric order step by step:

1. **Base Cost** = $8,000 \times \$4.50 = \$36,000$
2. **Dyeing Cost** = $8,000 \times \$0.40 = \$3,200$
3. **Finishing Cost** = $8,000 \times \$0.60 = \$4,800$
4. **Total before Shipping** = $\$36,000 + \$3,200 + \$4,800 = \$44,000$
5. **Final Cost** = $\$44,000 + \$800 = \$44,800$

So, the total cost for the order is **\$44,800**.

Prompt

Response



Chain-of-Thought (CoT) Prompting

5.3 Trigger

Task: Calculate the total cost for a bulk fabric order, considering different cost factors.

Fabric: 80% Polyester, 20% Cotton

Order Quantity: 8,000 meters

Base Price: \$4.50 per meter

Additional Costs:

- Dyeing: \$0.40 per meter
- Special Finishing: \$0.60 per meter
- Shipping: Flat \$800

Let's think step by step.

Let's break it down step by step:

1. Base Fabric Cost

$$8,000 \text{ meters} \times 4.50 \text{ per meter} = 36,000$$

2. Dyeing Cost

$$8,000 \text{ meters} \times 0.40 \text{ per meter} = 3,200$$

3. Special Finishing Cost

$$8,000 \text{ meters} \times 0.60 \text{ per meter} = 4,800$$

4. Shipping Cost (Flat fee)

$$800$$

5. Total Cost

$$36,000 + 3,200 + 4,800 + 800 = 44,800$$

Final Answer:

The total cost for the bulk fabric order is **\$44,800**.

Prompt

Response



Tree-of-Thought (ToT) Prompting

6.1 Định nghĩa

- Tree-of-Thought (ToT) Prompting mở rộng Chain-of-Thought (CoT) bằng cách khám phá nhiều lộ trình suy luận thay vì chỉ một lộ trình duy nhất.
- AI tạo ra một cấu trúc giống như cây với các nhánh suy nghĩ khác nhau trước khi chọn ra nhánh tốt nhất.

Tree-of-Thought (ToT) Prompting

6.2 Example

Nhiệm vụ: Thiết kế hệ thống nâng cấp vũ khí cho game nhập vai sao cho cân bằng giữa độ hiếm, rủi ro thất bại, và sự hấp dẫn với người chơi.

Bây giờ hãy tạo ra nhiều phương án khả thi, sau đó phân tích ưu nhược điểm của từng phương án trước khi đưa ra lựa chọn cuối cùng.

Prompt

✅ PHƯƠNG ÁN 1: Tỷ lệ thành công giảm dần theo cấp độ

Cách hoạt động:

- Vũ khí có cấp nâng từ +0 đến +10 (hoặc cao hơn).
- Tỷ lệ nâng cấp thành công giảm dần:
 - +1: 100%
 - +2: 95%
 - ...
 - +10: 20%
- Thất bại có thể gây:
 - Mất nguyên liệu
 - Giảm cấp nâng
 - Hoặc phá hủy vũ khí (tùy chọn)

Ưu điểm:

- Tạo cảm giác cân bằng giữa rủi ro và phần thưởng.
- Gây "nghiện" vì có cơ chế đỏ đen -> dễ thu hút người chơi.
- Có thể bán vật phẩm "bảo hiểm" để giảm rủi ro => thúc đẩy doanh thu (nếu có shop).

Nhược điểm:

- Thiếu kiểm soát: người chơi có thể thất bại liên tục và nản.
- Nếu không cẩn thận sẽ gây cảm giác "hút máu" nếu gắn liền với IAP.

✅ PHƯƠNG ÁN 2: Tích điểm nâng cấp (Pity System)

Cách hoạt động:

- Mỗi lần nâng cấp thất bại sẽ cộng điểm "pity".
- Khi đủ điểm, lượt tiếp theo chắc chắn thành công.
- Có thể hiển thị "% thành công" hiện tại tăng theo pity.

🎯 LỰA CHỌN CUỐI CÙNG (GỢI Ý PHỐI HỢP)

Một hệ thống hấp dẫn và cân bằng có thể kết hợp:

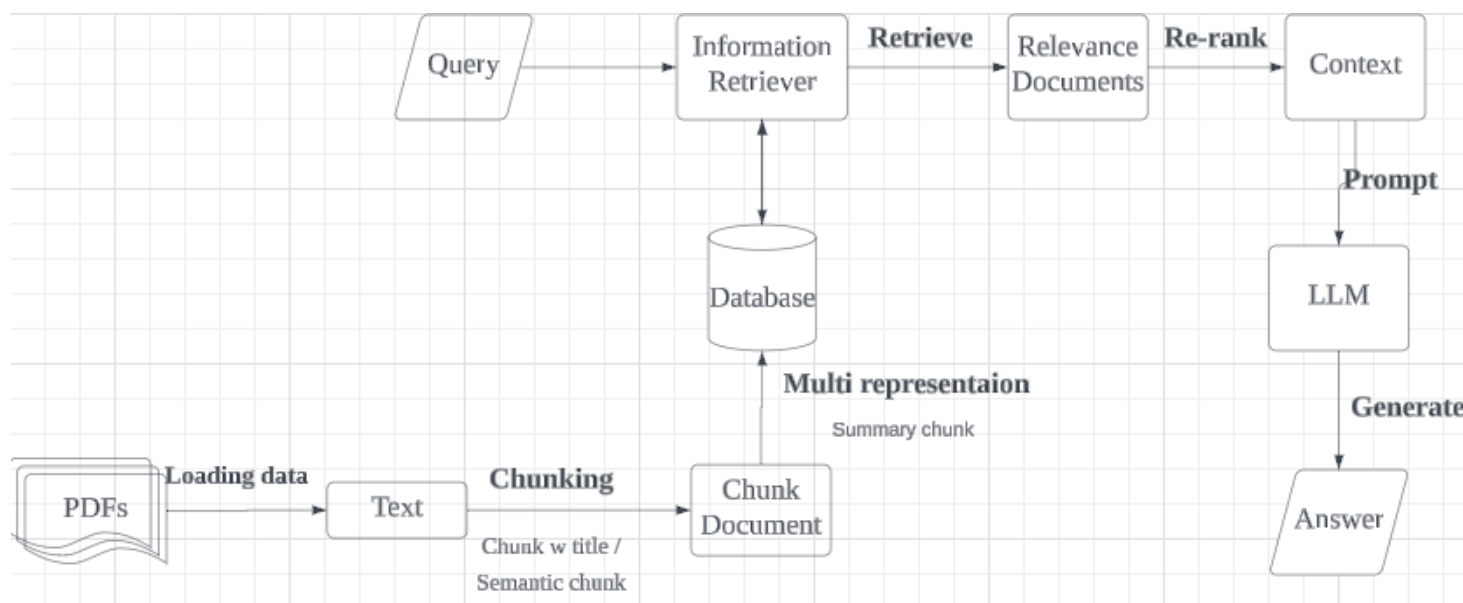
- Cơ chế nâng cấp chính: Tỷ lệ thành công giảm dần (Phương án 1)
- Pity system mềm: sau 3 lần thất bại thì lần 4 có bonus tỉ lệ (Phương án 2)
- Nguyên liệu tăng tỉ lệ: người chơi có thể dùng đá hiếm để gia tăng cơ hội (Phương án 3)
- Nhánh nâng cấp đặc biệt ở mốc +5, +10 (Phương án 4)

🌸 Việc kết hợp giúp thỏa mãn cả người chơi thích đỏ đen, chiến lược, và tùy biến.

Response

3. Ứng dụng bài toán Hỏi đáp về Luật

3.1 Pipeline



3. Ứng dụng bài toán Hỏi đáp về Luật

3.2 Loading data

- Sử dụng *partition_pdf*

```
{ "<class 'unstructured.documents.elements.Header'>": 1,  
  "<class 'unstructured.documents.elements.Title'>": 56,  
  "<class 'unstructured.documents.elements.NarrativeText'>": 421,  
  "<class 'unstructured.documents.elements.ListItem'>": 1,  
  "<class 'unstructured.documents.elements.Image'>": 1 }
```

- Lấy ra Title + NarrativeText

3. Ứng dụng bài toán Hỏi đáp về Luật

3.2 Chunking

- Tiền xử lý: Loại bỏ các đoạn text có kích thước < 10 hoặc không chứa kí tự

- **Semantic Chunking:**

- Kết hợp 3 câu $\Rightarrow$ 1 combine_sentences

- Tính Cosine-distance, percentile = 80

- **Chunk with title:**

- Mỗi chunk sẽ bao gồm cả Điều, Khoản, Điểm

```
{
  "id": null,
  "metadata": {},
  "page_content": "Điều 3. Giải thích từ ngữ 17. Phương tiện giao thông đường bộ gồm phương tiện giao thông cơ giới đường bộ, phương tiện giao thông thô sơ đường bộ. ",
  "type": "Document"
},
{
  "id": null,
  "metadata": {},
  "page_content": "Điều 3. Giải thích từ ngữ 18. Phương tiện giao thông cơ giới đường bộ (sau đây gọi là xe cơ giới) gồm xe ô tô; máy kéo; rơ moóc hoặc sơ mi rơ moóc được kéo bởi xe ô tô, máy kéo; xe mô tô hai bánh; xe mô tô ba bánh; xe gắn máy (kể cả xe máy điện) và các loại xe tương tự. ",
  "type": "Document"
},
```

3. Ứng dụng bài toán Hỏi đáp về Luật

3.4 Indexing

- Embedding model: *keepitreal/vietnamese-sbert*
 - Multi-representation indexing:
 - Tóm tắt các chunk
 - Lưu cả chunk gốc và bản tóm tắt vào database
- => Semantic search trên bản tóm tắt và ánh xạ trả về chunk gốc

3. Ứng dụng bài toán Hỏi đáp về Luật

3.5 Retriever & Re-ranker

- Retriever trả về top_k chunk liên quan query
- Re-ranker:
 - Model: '*namdp-ptit/viranker*'
 - Xếp hạng lại theo độ liên quan giảm dần.

3. Ứng dụng bài toán Hỏi đáp về Luật

3.6 Generater answer

- One-shot prompting (có thể dùng kỹ thuật khác)

```
template = """Bạn là chuyên gia về Luật Giao thông đường bộ Việt Nam. Hãy trả lời một cách chính xác, ngắn gọn không lặp lại thông tin và không tạo ra gì thêm ngoài câu trả lời.
### **Ví dụ**
**Câu hỏi:** "Quy định về chấp hành tín hiệu đèn giao thông là gì?"
**Trả lời:** "Tín hiệu đèn giao thông có ba màu: xanh (được đi), đỏ (dừng lại), vàng (chuẩn bị dừng, có thể đi nếu đã quá vạch dừng)." \n\n
---

Bây giờ, trả lời câu hỏi dựa trên ngữ cảnh sau:
{context}

{question}
**Trả lời:**
"""
```

